

Document technique SkyStorm

Guide debutant du projet Laravel realise pour un contexte BTS SIO

Date : 29/06/2026

Ce document explique le projet comme si le lecteur debutait en programmation. La premiere partie presente l'architecture generale du projet, les routes, la base de donnees et les dossiers importants. La seconde partie detaille les fonctionnalites une par une avec les fichiers et lignes utiles pour les retrouver rapidement dans le code source.

Projet : mini reseau social Laravel nomme SkyStorm, avec publications, abonnements, favoris, profils, moderation, notes et messagerie privatee.

1. Vue d'ensemble du projet

SkyStorm est une application web écrite avec le framework Laravel. Le but du projet est de proposer un réseau social simple, lisible et défendable à l'oral : l'utilisateur crée un compte, publie des messages, suit d'autres comptes, ajoute des favoris, commente, signale un contenu et peut discuter en privé avec d'autres membres.

La logique a volontairement été gardée simple : beaucoup de fonctionnalités sont gérées avec des contrôleurs courts, des vues Blade lisibles et des tables SQL bien séparées. C'est un bon format pour un examen, parce qu'on peut rapidement montrer le chemin complet d'une fonctionnalité : route -> contrôleur -> modèle -> vue -> base de données.

- Base technique : Laravel 12, PHP 8.2, Blade, migrations Eloquent, SQLite dans le projet actuel.
- Partie front : pages Blade avec Bootstrap et quelques styles personnalisés dans le layout.
- Build front : Vite charge les assets CSS, Sass et JavaScript.
- Logique métier : centralisée dans app/Http/Controllers/ et les relations des modèles Eloquent.

2. Rappel débutant : architecture MVC

MVC signifie Model - View - Controller. C'est une façon d'organiser le projet pour séparer les responsabilités.

Composant	Rôle simple	Exemples dans le projet
Model	Représente les données et les relations avec la base	app/Models/User.php, Post.php, Message.php
View	Affiche les pages HTML à l'utilisateur	resources/views/home.blade.php, posts/_card.blade.php
Controller	Reçoit la requête, applique les règles, choisit la vue	HomeController.php, PostController.php, MessageController.php
Route	Associe une URL à un contrôleur	routes/web.php

Exemple concret quand un utilisateur like un post : la route appelle LikeController, le contrôleur ajoute une relation dans la table post_likes, puis Laravel revient à la page précédente avec un message de succès.

Route du like : routes/web.php:51-52

Contrôleur du like : app/Http/Controllers/LikeController.php:9-20

Vue du bouton like : resources/views/posts/_card.blade.php:48-57

3. Organisation des dossiers

Pour un débutant, le plus important est de savoir où chercher. La structure ci-dessous montre les zones principales du projet.

```
SkyStorm-BTS26/
|- app/
|  |- Http/Controllers/    -> logique métier des routes
|  |- Models/              -> relations avec les tables
|- database/
|  |- migrations/          -> structure SQL du projet
|  |- factories/           -> objets de test
|- resources/views/        -> pages Blade (interface)
|- routes/web.php          -> déclaration des URL
|- tests/Feature/          -> tests fonctionnels
|- public/                 -> assets publics
|- output/pdf/             -> PDF final généré
`- tmp/pdfs/               -> fichiers temporaires pour la génération PDF
```

Si tu veux comprendre une fonctionnalité pendant l'oral, commence presque toujours par `routes/web.php`, puis ouvre le contrôleur cible, puis le modèle éventuel, puis la vue Blade.

4. Fichiers les plus importants

Routes principales - Toutes les URL et les actions disponibles.

Emplacement : `routes/web.php` : 14-64

Modèle utilisateur - Profil, relations followers, favoris, messages et helpers.

Emplacement : `app/Models/User.php` : 22-140

Modèle post - Relations du post et règle de visibilité pour les visiteurs.

Emplacement : `app/Models/Post.php` : 12-48

Layout commun - Navigation, topbar, messages flash et structure des pages.

Emplacement : `resources/views/layouts/app.blade.php` : 81-181

Accueil - Construction du feed et des statistiques.

Emplacement : `app/Http/Controllers/HomeController.php` : 15-49

Carte d'un post - Actions likes, favoris, commentaires, signalement.

Emplacement : `resources/views/posts/_card.blade.php` : 7-112

5. Les routes du projet

Les routes jouent le rôle d'annuaire de l'application. C'est elles qui disent : quand un navigateur demande une URL, quel contrôleur doit réagir ?

Zone	Exemples	Où regarder
Invites	/, /explore, /members	routes/web.php:14-23
Authentification	login, register, reset password	routes/web.php:25 et resources/views/auth/
Accueil connecté	/home	routes/web.php:27, app/Http/Controllers/HomeController.php:15-49
Posts	/posts, /posts/create	routes/web.php:29-35, app/Http/Controllers/PostController.php:15-80
Profil	/users/{user}, /profile/edit	routes/web.php:20-22 et 44-45
Interactions	likes, commentaires, favoris, signalements	routes/web.php:41-60
Messagerie	/messages, /messages/{user}	routes/web.php:47-49
Administration	/admin/reports	routes/web.php:62-63

Fichier central des routes : routes/web.php:18-64

6. La base de données

Laravel construit la base à partir des migrations. Une migration est un fichier PHP qui décrit une table SQL : ses colonnes, ses clés et ses contraintes. C'est pratique car le schéma de la base reste versionné avec le code.

Table	Rôle	Migration
users	Comptes, profil, statut admin, avatar, bio	create_users + add_social_fields + add_profile_fields
posts	Publications du réseau social	2026_02_12_185401_create_posts_table.php
followers	Relation abonnement abonné / suivi	2026_03_19_164551_create_followers_table.php
notes	Notes privées de l'utilisateur	2026_03_19_000000_create_notes_table.php
post_likes	Likes sur les publications	2026_06_29_120100_create_post_likes_table.php
favorite_posts	Publications mises en favoris	2026_06_29_120200_create_favorite_posts_table.php
comments	Commentaires sur un post	2026_06_29_120300_create_comments_table.php
post_reports	Signalements et modération	2026_06_29_120400_create_post_reports_table.php
messages	Messages privés entre deux utilisateurs	2026_06_29_120600_create_messages_table.php
sessions	Sessions Laravel des utilisateurs connectés	0001_01_01_000000_create_users_table.php

Users de base : database/migrations/0001_01_01_000000_create_users_table.php:14-37

Posts : database/migrations/2026_02_12_185401_create_posts_table.php:15-20

Followers : database/migrations/2026_03_19_164551_create_followers_table.php:11-25

Champs profil : database/migrations/2026_06_29_120500_add_profile_fields_to_users_table.php:11-16

Messages : database/migrations/2026_06_29_120600_create_messages_table.php:12-18

7. Cycle d une requete : exemple simple

Prenons un exemple concret : ajouter une publication aux favoris. Ce scenario aide a comprendre comment tout s enchaines dans Laravel.

1. L utilisateur clique sur le bouton dans la vue Blade du post.
2. Le formulaire envoie une requete POST vers la route `posts.favorites.store`.
3. La route appelle `FavoriteController@store`.
4. Le controleur applique la regle metier : maximum 50 favoris.
5. Le modele User ajoute la relation dans la table pivot `favorite\_posts`.
6. Laravel redirige avec un message de succes visible dans le layout.

Bouton dans la carte du post : `resources/views/posts/_card.blade.php:59-67`

Route POST : `routes/web.php:54-55`

Controleur FavoriteController : `app/Http/Controllers/FavoriteController.php:58-68`

Table pivot favorite_posts : `database/migrations/2026_06_29_120200_create_favorite_posts_table.php:13-20`

Message flash : `resources/views/layouts/app.blade.php:163-169`

8. Fiches fonctionnelles

Les fiches suivantes sont conçues pour la révision. Pour chaque fonctionnalité : tu trouves son but, sa logique simple et les fichiers utiles à ouvrir dans l'ordre.

Authentification

1. Inscription d'un utilisateur

Laravel UI fournit le formulaire, la validation et la création du compte. Le projet ajoute une petite règle utile : le tout premier compte créé devient administrateur.

Fonctionnalités associées : Cette fonctionnalité est liée à la connexion, au profil utilisateur et à la modération admin.

Où regarder dans le code :

Routes auth : `routes/web.php:25`

Contrôleur : `app/Http/Controllers/Auth/RegisterController.php:49-71`

Vue : `resources/views/auth/register.blade.php:11-71`

2. Connexion et déconnexion

Un utilisateur se connecte avec son email et son mot de passe. La session est ensuite gérée par Laravel, puis les pages protégées deviennent accessibles.

Fonctionnalités associées : Toutes les autres fonctionnalités sociales importantes dépendent de la connexion.

Où regarder dans le code :

Routes auth : `routes/web.php:25`

Vue login : `resources/views/auth/login.blade.php:11-67`

Bouton logout : `resources/views/layouts/app.blade.php:132-136`

Accueil

3. Tableau de bord après connexion

La page d'accueil connectée centralise le fil d'actualité, les statistiques, les notes récentes, les favoris et les suggestions de profils à suivre.

Fonctionnalités associées : C'est le point d'entrée principal du projet une fois l'utilisateur connecté.

Où regarder dans le code :

Route : `routes/web.php:27`

Contrôleur : `app/Http/Controllers/HomeController.php:15-49`

Vue accueil : `resources/views/home.blade.php:4-159`

4. Affichage des statistiques personnelles

Le projet calcule le nombre d'abonnés, de publications, de likes reçus, le meilleur score d'un post et le nombre de favoris. C'est directement exploitable pour un sujet d'examen sur les agrégats.

Fonctionnalités associées : Lié au modèle User, aux posts, aux likes et à la page d'accueil.

Où regarder dans le code :

Calcul des stats : `app/Http/Controllers/HomeController.php:39-47`

Affichage : `resources/views/home.blade.php:9-40`

Posts

5. Creation d une publication

L utilisateur saisit un texte court, le formulaire verifie la taille maximale puis le post est enregistre avec l identifiant du proprietaire.

Fonctionnalites associees : Cette fonctionnalite sert ensuite aux likes, commentaires, favoris, signalements et au profil.

Ou regarder dans le code :

Route resource : routes/web.php:33-35

Controleur store : app/Http/Controllers/PostController.php:38-46

Formulaire accueil : resources/views/home.blade.php:42-64

Formulaire detaille : resources/views/posts/create.blade.php:10-28

6. Modification d une publication

Seul l auteur du post peut ouvrir le formulaire puis mettre a jour le contenu. C est une bonne demonstration de controle d acces simple.

Fonctionnalites associees : Associe a la securite de base : un utilisateur ne doit pas modifier le contenu d un autre.

Ou regarder dans le code :

Controleur edit : app/Http/Controllers/PostController.php:49-55

Controleur update : app/Http/Controllers/PostController.php:57-69

Vue edit : resources/views/posts/edit.blade.php:10-28

Bouton dans la carte : resources/views/posts/_card.blade.php:26-30

7. Suppression d une publication

La suppression est reservee au proprietaire du post. Comme les relations SQL utilisent `cascadeOnDelete`, les enregistrements dependants comme likes ou commentaires peuvent etre geres proprement par la base.

Fonctionnalites associees : Fonction annexe utile a expliquer pour parler d integrite referentielle.

Ou regarder dans le code :

Controleur destroy : app/Http/Controllers/PostController.php:72-79

Bouton de suppression : resources/views/posts/_card.blade.php:30-36

Social

8. Fil d actualite base sur les abonnements

Le feed montre les publications des personnes suivies ainsi que celles de l utilisateur lui meme. On recupere d abord les identifiants suivis, puis on charge les posts correspondants.

Fonctionnalites associees : Lie a la table `followers`, au modele User et a la page d accueil.

Ou regarder dans le code :

Recuperation des ids suivis : app/Http/Controllers/HomeController.php:19-23

Table followers : database/migrations/2026_03_19_164551_create_followers_table.php:11-25

Relations User : app/Models/User.php:101-126

9. Suivre un utilisateur

Le controleur ajoute une relation dans la table pivot `followers`. Il evite aussi qu'un utilisateur se suive lui meme.

Fonctionnalites associees : Associe aux suggestions, aux profils et aux statistiques d'abonnes.

Ou regarder dans le code :

Route : routes/web.php:37

Controleur : app/Http/Controllers/FollowController.php:14-25

Bouton suggestions : resources/views/home.blade.php:129-152

10. Ne plus suivre un utilisateur

L'action retire la relation dans la table pivot. C'est le pendant logique du follow.

Fonctionnalites associees : Fonction annexe du systeme d'abonnements.

Ou regarder dans le code :

Route : routes/web.php:38

Controleur : app/Http/Controllers/FollowController.php:27-31

Bouton profil : resources/views/profiles/show.blade.php:43-54

Profils

11. Annuaire des membres

La page membre affiche la liste des utilisateurs avec une recherche par nom, email ou localisation. C'est simple mais tres typique d'un exercice de CRUD et de filtre.

Fonctionnalites associees : Fonction utile pour decouvrir des profils et lancer une conversation privatee.

Ou regarder dans le code :

Route : routes/web.php:19

Controleur : app/Http/Controllers/ProfileController.php:10-28

Vue : resources/views/profiles/index.blade.php:4-50

12. Consultation d'un profil public

Chaque utilisateur a une page profil avec ses informations, ses statistiques et ses publications. C'est une fonctionnalite centrale d'un reseau social.

Fonctionnalites associees : Elle se connecte aux followers, favoris publics, messagerie et posts.

Ou regarder dans le code :

Route : routes/web.php:20

Controleur : app/Http/Controllers/ProfileController.php:30-56

Vue : resources/views/profiles/show.blade.php:4-75

13. Listes des abonnées et des abonnements

Le profil permet d'ouvrir une liste des personnes qui suivent un compte et une autre liste des comptes suivis. C'est utile pour expliquer les relations many-to-many.

Fonctionnalites associees : Annexe directe du systeme de follow.

Ou regarder dans le code :

Routes : routes/web.php:21-22

Controleur followers : app/Http/Controllers/ProfileController.php:59-68

Controleur following : app/Http/Controllers/ProfileController.php:71-80

Vue liste : `resources/views/profiles/relations.blade.php:4-33`

14. Modification des informations du profil

Un utilisateur peut modifier son nom, son email, sa bio, sa localisation, son site web, sa date de naissance et éventuellement son mot de passe.

Fonctionnalités associées : Cette fonctionnalité est souvent attendue dans un sujet de gestion de compte utilisateur.

Où regarder dans le code :

Routes : `routes/web.php:44-45`

Contrôleur : `app/Http/Controllers/ProfileSettingsController.php:12-58`

Vue formulaire : `resources/views/profiles/edit.blade.php:13-68`

Colonnes SQL : `database/migrations/2026_06_29_120500_add_profile_fields_to_users_table.php:11-16`

15. Photo de profil (ajout, remplacement, suppression)

La photo est stockée dans le disque `public` de Laravel. Le contrôleur gère le cas où l'utilisateur remplace ou supprime son image actuelle.

Fonctionnalités associées : Annexe directe des réglages profil et visible dans l'interface.

Où regarder dans le code :

Validation et upload : `app/Http/Controllers/ProfileSettingsController.php:23-46`

Suppression d'avatar : `app/Http/Controllers/ProfileSettingsController.php:35-37`

Helper URL : `app/Models/User.php:133-139`

Composant avatar : `resources/views/users/_avatar.blade.php:1-15`

Interactions

16. Likes sur les publications

Le like est géré comme une table pivot `post\_likes`. Cela montre bien comment relier utilisateurs et publications sans dupliquer de colonnes dans `posts`.

Fonctionnalités associées : Lie au compteur de likes et aux statistiques du tableau de bord.

Où regarder dans le code :

Routes : `routes/web.php:51-52`

Contrôleur : `app/Http/Controllers/LikeController.php:9-20`

Modèle User : `app/Models/User.php:76-79`

Migration : `database/migrations/2026_06_29_120100_create_post_likes_table.php:13-20`

Boutons vue : `resources/views/posts/_card.blade.php:48-57`

17. Commentaires sur les publications

Les commentaires sont stockés dans une table séparée. L'auteur du commentaire, ou l'auteur du post, peut supprimer le commentaire.

Fonctionnalités associées : Fonction sociale classique, facile à expliquer avec une relation one-to-many.

Où regarder dans le code :

Routes : `routes/web.php:57-58`

Contrôleur : `app/Http/Controllers/CommentController.php:11-35`

Migration : `database/migrations/2026_06_29_120300_create_comments_table.php:13-18`

18. Ajout et retrait des favoris

Chaque utilisateur peut enregistrer des publications dans sa liste personnelle de favoris. La relation est gérée dans la table pivot `favorite_posts`.

Fonctionnalités associées : Lie aux favoris publics et a la limite de 50 éléments.

Ou regarder dans le code :

Routes : `routes/web.php`:41-42 et 54-55

Contrôleur : `app/Http/Controllers/FavoriteController.php`:11-85

Modèle User : `app/Models/User.php`:81-84

Migration : `database/migrations/2026_06_29_120200_create_favorite_posts_table.php`:13-20

Boutons : `resources/views/posts/_card.blade.php`:59-67

19. Limite de 50 favoris

Le contrôleur refuse l'ajout d'un nouveau favori si la liste contient déjà 50 publications. C'est un exemple très clair de règle métier.

Fonctionnalités associées : Fonction annexe du système de favoris.

Ou regarder dans le code :

Contrôle de la limite : `app/Http/Controllers/FavoriteController.php`:58-64

Test automatisé : `tests/Feature/SocialFeaturesTest.php`:16-40

20. Favoris publics ou privés

Un utilisateur choisit si sa liste de favoris reste privée ou devient visible publiquement. Cette information est stockée dans `users.favorite_posts_public`.

Fonctionnalités associées : Lie à la page `Mes favoris` et à la consultation publique des favoris d'un autre profil.

Ou regarder dans le code :

Mise à jour de visibilité : `app/Http/Controllers/FavoriteController.php`:78-85

Vue personnelle : `resources/views/favorites/index.blade.php`:5-29

Vue publique : `resources/views/favorites/show.blade.php`:4-14

Champ SQL : `database/migrations/2026_06_29_120000_add_social_fields_to_users_table.php`:11-14

Navigation

21. Explorer et rechercher des publications

La page Explorer permet de chercher un texte dans le contenu d'un post ou dans le nom de son auteur. Elle est utile même pour les visiteurs non connectés.

Fonctionnalités associées : Elle se combine avec la moderation, car certains posts peuvent être masqués en mode visiteur.

Ou regarder dans le code :

Route : `routes/web.php`:18

Contrôleur : `app/Http/Controllers/ExploreController.php`:10-40

Vue : `resources/views/explore.blade.php`:4-26

Champ recherche global : `resources/views/layouts/app.blade.php`:145-150

Moderation

22. Signaler une publication

Un utilisateur peut envoyer un signalement avec une raison. Le code empêche d'envoyer plusieurs signalements en attente sur le même post par la même personne.

Fonctionnalités associées : Lie au panneau d'administration et au masquage des posts pour les visiteurs.

Où regarder dans le code :

Route : `routes/web.php:60`

Contrôleur : `app/Http/Controllers/ReportController.php:10-31`

Migration : `database/migrations/2026_06_29_120400_create_post_reports_table.php:13-21`

Formulaire vue : `resources/views/posts/_card.blade.php:105-110`

23. Moderation admin des signalements

Un administrateur consulte la liste des signalements puis choisit de les accepter ou de les rejeter. Le premier utilisateur du projet peut devenir admin automatiquement.

Fonctionnalités associées : Très pratique à expliquer pour parler de rôle, autorisation et cycle de traitement.

Où regarder dans le code :

Routes : `routes/web.php:62-63`

Contrôleur admin : `app/Http/Controllers/AdminReportController.php:10-41`

Vue admin : `resources/views/admin/reports/index.blade.php:4-58`

Premier admin : `app/Http/Controllers/Auth/RegisterController.php:64-71`

24. Masquer les posts signalés aux visiteurs

Quand un visiteur non connecté consulte Explorer ou des favoris publics, les publications ayant un signalement en attente ou accepté sont exclues.

Fonctionnalités associées : Bonne illustration d'une règle métier implémentée à la fois dans le modèle et dans les contrôleurs.

Où regarder dans le code :

Scope du modèle : `app/Models/Post.php:43-47`

Utilisation dans Explorer : `app/Http/Controllers/ExploreController.php:25-27`

Utilisation dans favoris publics : `app/Http/Controllers/FavoriteController.php:47-53`

Test automatisé : `tests/Feature/SocialFeaturesTest.php:42-80`

Productivité

25. Notes privées

Le projet inclut aussi un mini module de notes privées. C'est très bien pour montrer un CRUD complet simple, même si ce n'est pas le cœur du réseau social.

Fonctionnalités associées : Annexe utile pour montrer la maîtrise du pattern CRUD.

Où regarder dans le code :

Routes ressource : `routes/web.php:29-31`

Contrôleur : `app/Http/Controllers/NoteController.php:15-75`

Table notes : `database/migrations/2026_03_19_000000_create_notes_table.php:12-17`

Bloc accueil : `resources/views/home.blade.php:80-115`

Messagerie

26. Messagerie privée entre deux utilisateurs

Deux utilisateurs peuvent ouvrir une conversation puis s'envoyer des messages texte simples. Les messages sont stockés dans une table dédiée `messages`.

Fonctionnalités associées : Lie au profil public, à l'annuaire des membres et au compteur de messages non lus.

Où regarder dans le code :

Routes : `routes/web.php:47-49`

Contrôleur liste + conversation : `app/Http/Controllers/MessageController.php:11-79`

Modèle : `app/Models/Message.php:8-32`

Migration : `database/migrations/2026_06_29_120600_create_messages_table.php:12-18`

Vue liste : `resources/views/messages/index.blade.php:4-36`

Vue conversation : `resources/views/messages/show.blade.php:4-48`

27. Compteur de messages non lus

Le modèle User calcule le nombre de messages reçus qui n'ont pas encore de date `read\_at`. Cette valeur est affichée dans la navigation et dans la liste des conversations.

Fonctionnalités associées : Sous-fonctionnalité de la messagerie, intéressante pour parler d'état d'un message.

Où regarder dans le code :

Compteur dans User : `app/Models/User.php:128-130`

Navigation : `resources/views/layouts/app.blade.php:112-117` et `152-158`

Liste des conversations : `resources/views/messages/index.blade.php:26-30`

28. Lecture automatique d'une conversation

Lorsqu'on ouvre une conversation, les messages reçus dans cette discussion passent en `read\_at = now()`. C'est simple et pédagogique pour expliquer l'évolution d'un état.

Fonctionnalités associées : Sous-fonctionnalité de la messagerie et du compteur non lu.

Où regarder dans le code :

Marquage comme lu : `app/Http/Controllers/MessageController.php:48-51`

Affichage de l'état : `resources/views/messages/show.blade.php:27-31`

9. Tests et verification

Un petit fichier de tests a ete ajoute pour verifier quelques regles metier importantes. Dans un oral, cela montre que le projet n est pas seulement code, mais aussi verifie.

Tests fonctionnels : tests/Feature/SocialFeaturesTest.php:12-132

- La limite de 50 favoris est testee.
- Le masquage des posts signales pour les visiteurs est teste.
- Le traitement d un signalement par un admin est teste.
- L envoi d un message prive est teste.

10. Commandes utiles pour faire tourner le projet

Pour un examinateur, il est souvent utile de montrer que tu sais lancer le projet et expliquer les commandes de base.

```
php artisan migrate
```

```
php artisan storage:link
```

```
npm run app
```

```
php artisan test
```

Pourquoi `storage:link` ? Parce que la photo de profil utilise le disque public de Laravel. Le lien symbolique permet au navigateur d acceder aux fichiers stockes dans `storage/app/public`.

11. Conseils pour presenter le projet a l oral

- Commence par l architecture MVC avant de parler des details.
- Montre une fonctionnalite de bout en bout : route, controleur, modele, vue, table SQL.
- Utilise la messagerie, les favoris ou le signalement comme exemples, car le chemin technique y est tres clair.
- Si on te demande une regle metier, cite la limite de 50 favoris ou le filtrage des posts signales pour les visiteurs.
- Si on te demande la securite, cite l authentication, les controles d auteur sur les posts et la verification du role admin.

Fin du document. Ce support est volontairement detaille pour pouvoir servir a la fois de fiche de revision et de guide de navigation dans le code source.